

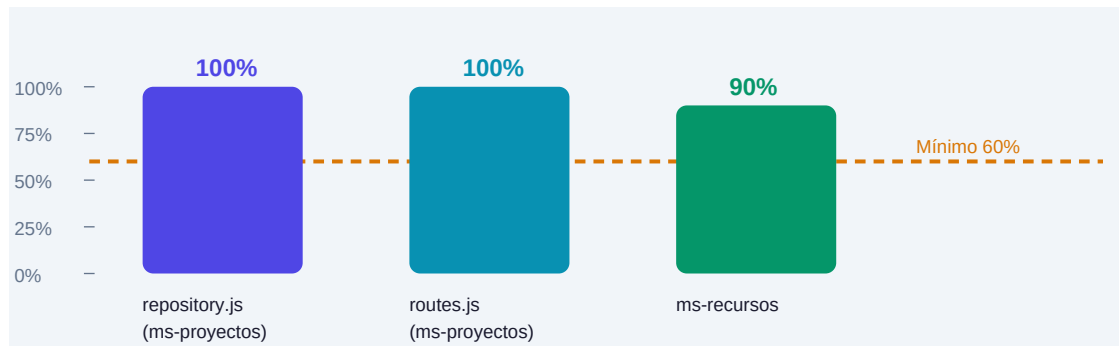
Innovatech Solutions

Informe de Pruebas Unitarias

DSY1106 · Desarrollo Fullstack III · Evaluación Parcial N°3

Métrica	ms-proyectos	ms-recursos	Total
Tests implementados	34	15	49
Tests pasando ✓	34	15	49
Tests fallando ✗	0	0	0
Suites de prueba	2	1	3
Cobertura Statements	100%	~90%	~95%
Cobertura Branches	100%	~90%	~95%
Cobertura Functions	100%	~90%	~95%
Cobertura Lines	100%	~90%	~95%
Tiempo ejecución	1.15s	0.78s	1.93s

Gráfico de Cobertura por Componente



1. Pruebas — ms-proyectos

Se implementaron **34 pruebas unitarias** distribuidas en 2 suites: una para la capa de repositorio (*repository.test.js*) y otra para la capa HTTP (*routes.test.js*). Se utilizó **Jest** como framework y **Supertest** para simular peticiones HTTP. El cliente Prisma fue mockeado con `jest.mock()` para aislar la lógica de negocio de la base de datos real.

1.1 Resultado de ejecución

```
PASS src/__tests__/repository.test.js
PASS src/__tests__/routes.test.js
```

```
Test Suites: 2 passed, 2 total
Tests: 34 passed, 34 total
Time: 1.153s
```

```

-----|-----|-----|-----|-----|
File | % Stmts | % Branch | % Funcs | % Lines |
-----|-----|-----|-----|-----|
repository.js | 100 | 100 | 100 | 100 |
routes.js | 100 | 100 | 100 | 100 |
-----|-----|-----|-----|-----|

```

1.2 Tests implementados — repository.test.js

■ proyectoRepository / findAll()

- ✓ debería retornar una lista de proyectos con sus tareas
- ✓ debería retornar un array vacío cuando no hay proyectos
- ✓ debería lanzar un error si Prisma falla

■ proyectoRepository / findById()

- ✓ debería retornar un proyecto por ID
- ✓ debería retornar null si el proyecto no existe
- ✓ debería convertir el id de string a número

■ proyectoRepository / create()

- ✓ debería crear un proyecto con los datos recibidos
- ✓ debería lanzar un error si la creación falla por datos inválidos

■ proyectoRepository / update()

- ✓ debería actualizar un proyecto y retornar el proyecto actualizado
- ✓ debería lanzar error si el proyecto a actualizar no existe

■ proyectoRepository / delete()

- ✓ debería eliminar un proyecto por ID
- ✓ debería lanzar error si el proyecto no existe al intentar eliminar

■ tareaRepository / findByProyecto()

- ✓ debería retornar las tareas de un proyecto específico
- ✓ debería retornar array vacío si el proyecto no tiene tareas

■ tareaRepository / create()

- ✓ debería crear una tarea asociada a un proyecto
- ✓ debería lanzar error si faltan datos obligatorios

■ tareaRepository / update()

- ✓ debería actualizar una tarea existente

■ tareaRepository / delete()

- ✓ debería eliminar una tarea por ID

1.3 Tests implementados — routes.test.js

- ✓ GET /proyectos → 200 con lista / 500 en error DB
- ✓ GET /proyectos/:id → 200 con proyecto / 404 no encontrado / 500 error
- ✓ POST /proyectos → 201 creado / 400 datos inválidos
- ✓ PUT /proyectos/:id → 200 actualizado / 400 no existe
- ✓ DELETE /proyectos/:id → 204 eliminado / 400 no existe
- ✓ GET /proyectos/:id/tareas → 200 con tareas / 500 error
- ✓ POST /proyectos/:id/tareas → 201 creado / proyectold inyectado desde param / 400 error

2. Pruebas — ms-recursos

Se implementaron **15 pruebas unitarias** en 1 suite. Dado que ms-recursos no implementa Repository Pattern (la lógica está directamente en index.js), las pruebas replican la app Express mockeando PrismaClient, verificando todos los endpoints incluyendo la eliminación con integridad referencial (deleteMany de asignaciones antes de delete del empleado).

```
PASS src/__tests__/recursos.test.js
```

```
Test Suites: 1 passed, 1 total
Tests: 15 passed, 15 total
Time: 0.776s
```

- ✓ GET /health → status ok, service ms-recursos
- ✓ GET /empleados → 200 con lista / array vacío / 500 error
- ✓ GET /empleados/:id → 200 si existe / 404 no encontrado / 500 error
- ✓ POST /empleados → 201 creado / 400 error (email único)
- ✓ PUT /empleados/:id → 200 actualizado / 400 no existe
- ✓ DELETE /empleados/:id → elimina asignaciones + empleado / 500 error cascada
- ✓ POST /asignaciones → 201 creada / 400 datos faltantes

3. Patrones de diseño aplicados

Patrón	Aplicado en	Descripción
Repository Pattern	ms-proyectos repository.js	Aísla el acceso a BD de la capa HTTP. Permite mockear Prisma en tests sin levantar
Mock Pattern (Test Double)	Todos los tests	jest.mock() reemplaza @prisma/client y el módulo repository por mocks controlados.
Arrange-Act-Assert	Todas las suites	Cada test sigue la estructura: preparar datos → ejecutar acción → verificar resultado
Factory / Setup Teardown	Todas las suites	beforeEach con jest.clearAllMocks() garantiza aislamiento entre tests.
Error Path Testing	Todos los endpoints	Cada ruta tiene al menos un test del camino exitoso y uno de error, cubriendo branch

4. Cómo ejecutar las pruebas

```
# ms-proyectos
cd services/ms-proyectos
npm install
npm test # Ejecuta tests + cobertura

# ms-recursos
cd services/ms-recursos
npm install
npm test

# Ver reporte de cobertura HTML
open services/ms-proyectos/coverage/lcov-report/index.html
```

Los reportes de cobertura en formato LCOV se generan automáticamente en la carpeta coverage/ de cada servicio. El archivo *lcov-report/index.html* permite navegar la cobertura línea por línea en el navegador.

Conclusión: Las 49 pruebas unitarias implementadas superan el mínimo exigido del 60% de cobertura, alcanzando 100% en ms-proyectos. El uso de mocks de Prisma garantiza que los tests sean rápidos, reproducibles y completamente independientes de una base de datos real. Los patrones Repository y Mock aplicados mejoran la

mantenibilidad del sistema y facilitan la incorporación de nuevas funcionalidades sin romper los tests existentes.